Exercise Sheet 8

Small-World Networks

5942UE Network Science

26 March 2026

Prof. Dr. Michael Granitzer

8.A The Small-World Concept

Exercise 8.A.1: Degrees of Separation — Estimation

A social network has $N = 10^9$ users and average degree $k = 200$.

1. Estimate average path length using $d \approx \log(N) / \log(k)$.
2. Compare to Milgram's ≈ 6 for the US population ($\approx 2 \times 10^8$).
3. When are real networks shorter or longer than the random baseline?
4. Recompute d if average degree drops to $k = 10$.

Solution:

1. $d \approx \log(10^9)/\log(200) \approx 20.7/5.3 \approx 3.9$ hops.

2. With $N = 2 \times 10^8$ the formula gives ≈ 3.6. Milgram observed ≈ 5–6. Real networks are *not* random — local clustering creates "dead ends" that lengthen routing chains ([?EasleyKleinberg2010]).

3. Real networks are *shorter* than random when hubs act as super-shortcuts; *longer* when high clustering traps walks within communities.

4. $d \approx \log(10^9)/\log(10) = 9$. Reducing degree from 200 to 10 more than doubles the expected path length.

Exercise 8.A.2: Is Karate Club a Small World?

Using `nx.karate_club_graph()`:

1. Compute average clustering C and average shortest path length L.
2. Generate a random graph with the same n and m. Compute C_{rand}, L_{rand}.
3. Compute the **small-world index** $\sigma = (C/C_{rand})/(L/L_{rand})$.
4. Compare to a pure ring lattice. What pattern emerges?

Solution:

```python
import networkx as nx

G = nx.karate_club_graph()
C_real = nx.average_clustering(G)
L_real = nx.average_shortest_path_length(G)

R = nx.gnm_random_graph(len(G), G.number_of_edges(), seed=42)
C_rand = nx.average_clustering(R)
L_rand = nx.average_shortest_path_length(R)

sigma = (C_real / C_rand) / (L_real / L_rand)
```

The karate club has $C/C_{rand} \approx 4$ but $L/L_{rand} \approx 1$ — much higher clustering than random, similar path length. A high σ (typically > 3) is the small-world signature: locally cliquey like a lattice, globally compact like a random graph.

8.B Rewiring and Path Lengths

Exercise 8.B.1: The Rewiring Intuition

A 20-node ring lattice with 4 nearest-neighbour connections each. Average path ≈ 5, clustering ≈ 0.5.

1. What is the maximum reduction in path length from rewiring **one** edge?
2. Why does rewiring destroy clustering more slowly than it shortens paths?
3. Sketch $C(p)$ and $L(p)$ as p goes from 0 to 1. Where is the "sweet spot"?
4. What does this say about real social networks?

Solution:

1. A single shortcut connecting maximally distant nodes (distance $\approx N/2 = 10$) can roughly halve the diameter — one shortcut has a disproportionately large global effect.

2. Clustering measures local triangle density; one rewired edge breaks at most a few triangles, while leaving most local structure intact. Path length is a global property highly sensitive to a single bridge.

3. $L(p)$ drops steeply around $p \sim 0.01$; $C(p)$ stays near $C(0)$ until moderate p. The **sweet spot** is $p \in [0.01, 0.1]$ — the small-world regime.

4. Real social networks sit firmly in this regime: high local clustering with rare long-range weak ties ([? EasleyKleinberg2010]).

Exercise 8.B.2: Watts–Strogatz Sweep

Sweep p from 0.001 to 1.0 (log-spaced) for `nx.watts_strogatz_graph(n=100, k=6, p)`:

1. Compute normalised $C(p)/C_0$ and $L(p)/L_0$.
2. Plot both against $\log(p)$.
3. Identify the small-world regime.
4. Is the transition sharp or gradual?

Solution:

```
import networkx as nx
import numpy as np

n, k = 100, 6
ps = np.logspace(-3, 0, 20)
G0 = nx.watts_strogatz_graph(n, k, 0)
C0 = nx.average_clustering(G0)
L0 = nx.average_shortest_path_length(G0)

results = []
for p in ps:
    G = nx.watts_strogatz_graph(n, k, p, seed=42)
    results.append((nx.average_clustering(G) / C0,
                    nx.average_shortest_path_length(G) / L0))
```

$L(p)$ collapses sharply even at $p \approx 0.01$, while $C(p)$ stays high until much larger p. The "sweet spot" $p \in [0.01, 0.1]$ is where the network is locally dense and globally compact.

8.C Search and Navigation

Exercise 8.C.1: Why Decentralized Search Works

A 16-node grid. Node $(1, 1)$ wants to route to $(4, 4)$.

1. Why does a greedy router stall when long-range links are **uniformly random**?
2. Why does Kleinberg's **inverse-square** rule ($\Pr \propto 1/d^2$) enable efficient routing?
3. What was Milgram's actual chain completion rate?
4. What's the difference between "short paths exist" and "short paths are *findable*"?

Solution:

1. Uniform shortcuts are not matched to the *scale* of the remaining distance — once near the target, the router has no link that bridges a small-but-not-tiny gap, and oscillates or stalls.

2. Inverse-square gives a **multi-scale** distribution: at every remaining distance, the probability of finding a link that halves it is roughly constant. Greedy routing terminates in $O(\log^2 N)$ steps. No other exponent works ([?EasleyKleinberg2010]).

3. Only $\approx 29\%$ of Milgram's chains completed. The "6 degrees" figure is the average over *successful* chains.

4. A random graph almost certainly contains short paths (a global structural property), but local actors only see neighbours. Kleinberg's result: short paths are necessary but not sufficient for decentralised navigability.

Exercise 8.C.2: Greedy Routing on Watts–Strogatz

Using `nx.watts_strogatz_graph(n=100, k=4, p=0.05)`:

1. For two distant nodes, find the BFS shortest-path length.
2. Simulate **greedy routing**: at each step move to the neighbour with smallest BFS distance to target.
3. Repeat 100 times with random pairs. Compare BFS vs. greedy.
4. What fraction of greedy routes fail (loop or exceed $2\times$ BFS)?

Solution:

```python
import networkx as nx
import random

G = nx.watts_strogatz_graph(100, 4, 0.05, seed=42)

def greedy_route(G, src, tgt, cap=200):
    dist = nx.single_source_shortest_path_length(G, tgt)
    cur, path = src, [src]
    for _ in range(cap):
        if cur == tgt:
            return path
        cur = min(G.neighbors(cur), key=lambda n: dist.get(n, float("inf")))
        if cur in path:
            return None
        path.append(cur)
    return None
```

Greedy routing usually succeeds but typically takes 1–3 hops more than the BFS optimum. In a Watts–Strogatz small world, short paths *exist* but local actors often take suboptimal routes — Kleinberg's multi-scale ties are needed for true navigability.



8.D Web Bow-Tie

Exercise 8.D.1: Web Core and Periphery

Directed graph, 9 nodes:

- SCC core: $A \to B \to C \to A$
- IN: $D \to A, E \to B$
- OUT: $C \to F, B \to G$
- Tendril: $D \to H$
- Isolated: I

1. Identify all strongly connected components.
2. Classify each node (SCC / IN / OUT / tendril / isolated).
3. Which nodes can reach each other?
4. What does IN vs. OUT vs. SCC mean for a web page?

Solution:

1. One non-trivial SCC: $\{A, B, C\}$. Singleton SCCs for every other node.

2. A,B,C: SCC core. D, E: IN. F, G: OUT. H: tendril (reachable only from D). I: isolated.

3. $D, E \rightarrow$ SCC; SCC $\rightarrow F, G$; $D \rightarrow H$ (dead end); D and E cannot reach each other; I cannot reach or be reached.

4. Web meaning:

- **IN:** new or poorly-linked pages that link *to* hubs but receive no links back.
- **OUT:** pages the SCC links to but which don't link back (dead-end resources).
- **SCC core:** mutually reachable hub of major sites.
- **Tendrils:** isolated sub-webs reachable only from the IN component.

Exercise 8.D.2: Bow-Tie in Python

1. Find all SCCs with `nx.strongly_connected_components(W)`.
2. Build the condensation graph: `nx.condensation(W)`.
3. Classify each SCC by position relative to the giant SCC.
4. What does IN-component membership mean for discoverability?

Solution:

```
import networkx as nx

W = nx.DiGraph()
W.add_edges_from([("A","B"),("B","C"),("C","A"),
                  ("D","A"),("E","B"),
                  ("C","F"),("B","G"),
                  ("D","H")])
W.add_node("I")

sccs = list(nx.strongly_connected_components(W))
C = nx.condensation(W, scc=sccs)
giant = max(range(len(sccs)), key=lambda i: len(sccs[i]))

reachable_from_giant = nx.descendants(C, giant) | {giant}
can_reach_giant = nx.ancestors(C, giant) | {giant}
```

IN-component pages can reach the SCC but not vice versa. OUT pages are reachable from the SCC but lead nowhere. Pages in IN components or tendrils are invisible to crawlers starting from the SCC core — link-following crawls leave large parts of the web unmapped.



Wrap-Up

- $d \approx \log N / \log k$ gives the order-of-magnitude small-world estimate
- a tiny rewiring fraction destroys global distance without destroying local clustering
- short paths existing $\neq$ short paths being *findable* by local actors
- directed web structure has fundamentally different reachability than undirected social graphs